

EduGenie – AI Powered Learning Assistant

Project Description

EduGenie is an AI-powered educational assistant developed to simplify learning using Generative Artificial Intelligence. The application assists students, self-learners, and educators by providing intelligent educational support through a user-friendly web interface.

The system is developed using Python and FastAPI for the backend, while HTML, CSS, and Jinja2 are used to build the frontend interface. Google Gemini API powers the AI functionalities by generating accurate educational responses based on user requests. SQLAlchemy is used as the Object Relational Mapper (ORM), and SQLite serves as the backend database for storing user details, user queries, and AI-generated responses.

EduGenie provides five major educational features:

- Intelligent Question Answering
- Simplified Concept Explanation
- AI-Powered Quiz Generation
- Educational Text Summarization
- Personalized Learning Path Recommendation

The application follows a modular architecture where each feature is implemented as a separate AI module, making the system scalable, maintainable, and easy to extend with additional functionalities in the future.

Scenarios

Scenario 1 – Intelligent Question Answering

A student enters an academic question through the EduGenie interface. The system processes the query using the Google Gemini API and generates an accurate and meaningful response. The interaction is stored in the SQLite database for future reference.

Scenario 2 – AI-Based Quiz Generation

A learner selects the Quiz Generation module and enters a topic such as Python Programming or Machine Learning. EduGenie automatically generates quiz questions that help users practice concepts and evaluate their understanding.

Scenario 3 – Educational Summarization and Learning Path Recommendation

A user submits educational content or enters a learning topic. EduGenie summarizes lengthy educational content into concise points and recommends a structured learning path, enabling users to learn concepts in the correct sequence.

Technical Architecture

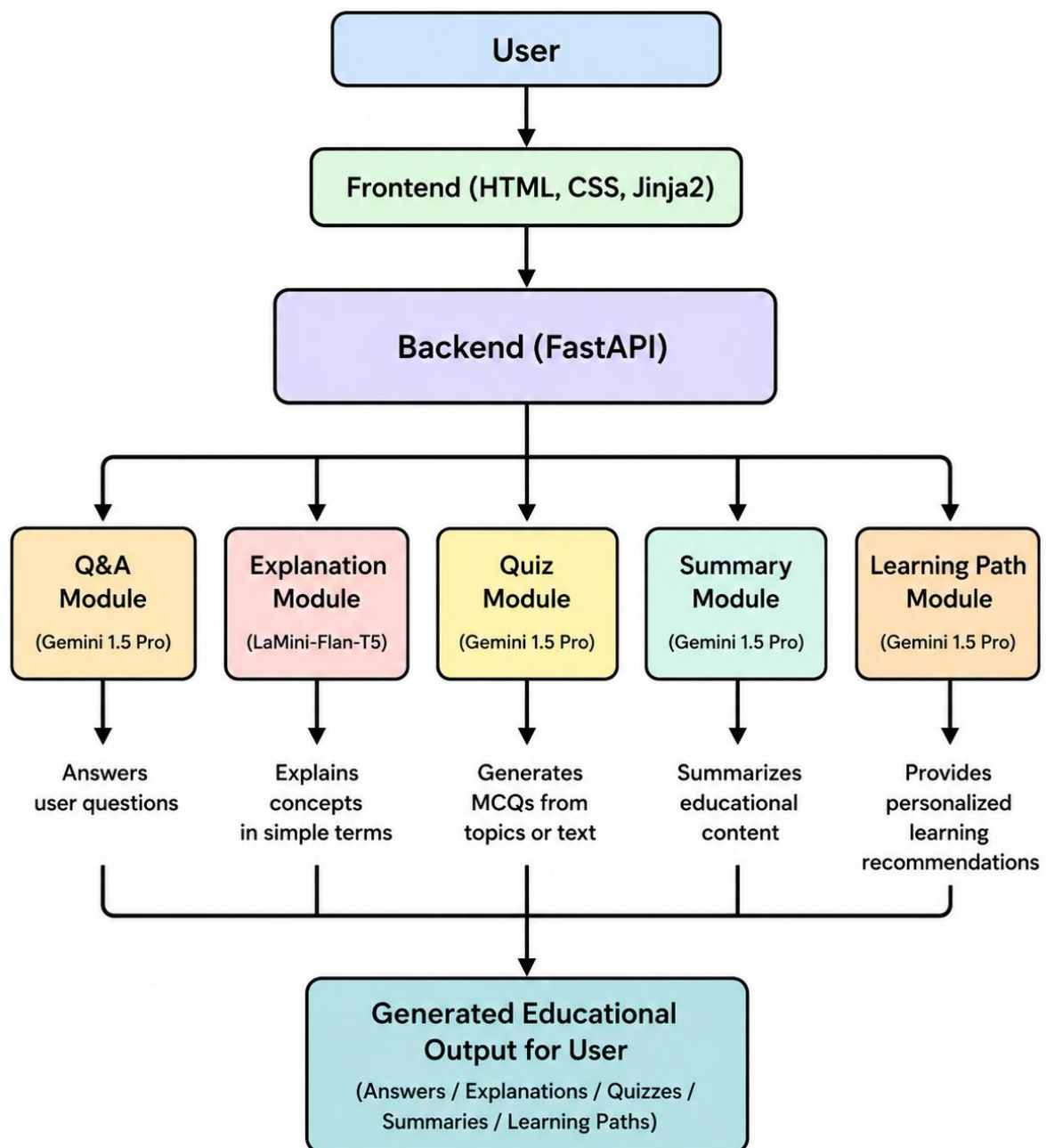
EduGenie follows a modular three-layer architecture consisting of the Presentation Layer, Backend Layer, AI Processing Layer, and Database Layer. The architecture is designed to provide an efficient and scalable educational assistant that delivers AI-generated learning content through multiple educational modules.

The workflow begins when the user interacts with the web interface developed using HTML, CSS, and Jinja2. User requests are sent to the FastAPI backend, which routes the request to the appropriate AI module based on the selected task.

The application consists of five independent AI modules:

- **Question Answering Module** – Generates accurate answers to user questions using the Google Gemini API.
- **Explanation Module** – Provides simplified explanations of concepts using the LaMini-Flan-T5 model.
- **Quiz Generation Module** – Creates quiz questions from the selected topic using the Google Gemini API.
- **Summarization Module** – Produces concise summaries of educational content using the Google Gemini API.
- **Learning Path Recommendation Module** – Suggests a structured learning path based on the user's selected topic using the Google Gemini API.

The generated responses are displayed through the frontend and are stored in the SQLite database using SQLAlchemy for future reference and analysis



Prerequisites

The following software and tools are required to develop and run the EduGenie application.

Software Requirements

- **Operating System:** Windows 10/11

- Python 3.10 or above
- Visual Studio Code
- Git
- Web Browser (Google Chrome or Microsoft Edge)

Python Libraries

- FastAPI
- Uvicorn
- SQLAlchemy
- Jinja2
- Google Generative AI
- Python-dotenv
- SQLite
- Python-Multipart

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB (8 GB Recommended)
- Storage: Minimum 2 GB Free Space
- Internet Connection (Required for Google Gemini API)

Project Workflow

The EduGenie application follows a structured workflow to process user requests and generate AI-powered educational content.

1. The user opens the EduGenie web application.
2. The user selects one of the available learning modules.
3. The user enters a topic or question.

4. The frontend sends the request to the FastAPI backend.
5. The backend routes the request to the appropriate AI module.
6. The selected module processes the request using Google Gemini API or LaMini-Flan-T5.
7. The generated response is stored in the SQLite database.
8. The frontend displays the generated educational content to the user.

Database Design

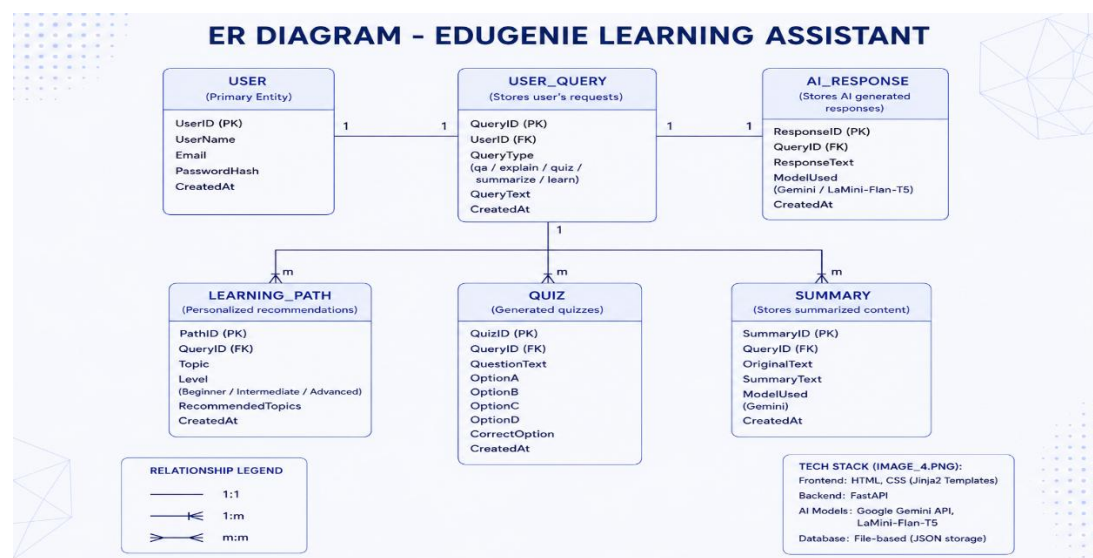
EduGenie uses SQLite as the backend database and SQLAlchemy as the Object Relational Mapper (ORM). The database stores user interactions and AI-generated responses.

The database contains the following tables:

- User
- UserQuery
- AIResponse

These tables help maintain user information, store submitted queries, and record AI-generated educational responses for future reference.

ER Diagram



Project Testing

The EduGenie application was tested to verify that all implemented features function correctly. Each module was executed with different educational topics and user inputs to ensure accurate AI-generated responses.

The following modules were tested successfully:

- Intelligent Question Answering
- Simplified Concept Explanation
- AI-Powered Quiz Generation
- Educational Text Summarization
- Personalized Learning Path Recommendation

The application successfully accepted user input, processed requests using the FastAPI backend, generated AI responses through the integrated models, and displayed the results on the web interface. The SQLite database also stored user queries and AI responses successfully.

Deployment

The EduGenie application was successfully deployed in a local development environment using FastAPI and Uvicorn. The project was developed in Visual Studio Code and tested through a web browser.

Deployment Steps

1. Install Python and create a virtual environment.
2. Install the required dependencies using the requirements.txt file.
3. Configure the Google Gemini API key in the .env file.
4. Run the application using the following command:

```
uvicorn app.main:app --reload
```

5. Open the application in the browser:

```
http://127.0.0.1:8000
```

The application successfully loaded all modules and generated AI-powered educational responses.

Validation

The EduGenie application was validated by testing each implemented module with different educational topics and user queries. The generated responses were verified for correctness, relevance, and usability.

The validation process confirmed that:

- The Question Answering module generated meaningful responses.
- The Concept Explanation module produced simple and understandable explanations.
- The Quiz Generation module created topic-based quiz questions.
- The Summarization module generated concise summaries.
- The Learning Path Recommendation module suggested structured learning sequences.

The SQLite database successfully stored user information, queries, and AI-generated responses.

Conclusion

EduGenie is an AI-powered educational assistant that simplifies the learning process through intelligent educational support. By integrating FastAPI, Google Gemini API, SQLAlchemy, and SQLite, the application provides accurate question answering, concept explanation, quiz generation, text summarization, and personalized learning path recommendations through a simple web interface.

The modular design of the application makes it scalable, maintainable, and suitable for future enhancements such as multilingual support, voice-based learning, student progress tracking, and Learning Management System (LMS) integration.